Foundations of Machine Learning and Deep Learning

13. Recommender systems

University Master's Degree in
Machine Learning

upna
Universidad Pública de Navarra
Nafarroako Unibertsitate Publikoa

Outline

- Introduction
- Content-based recommender systems
- Collaborative filtering-based recommender systems
  - User – user
  - Item – item
  - Latent factors
- Evaluation

Introduction



Client X

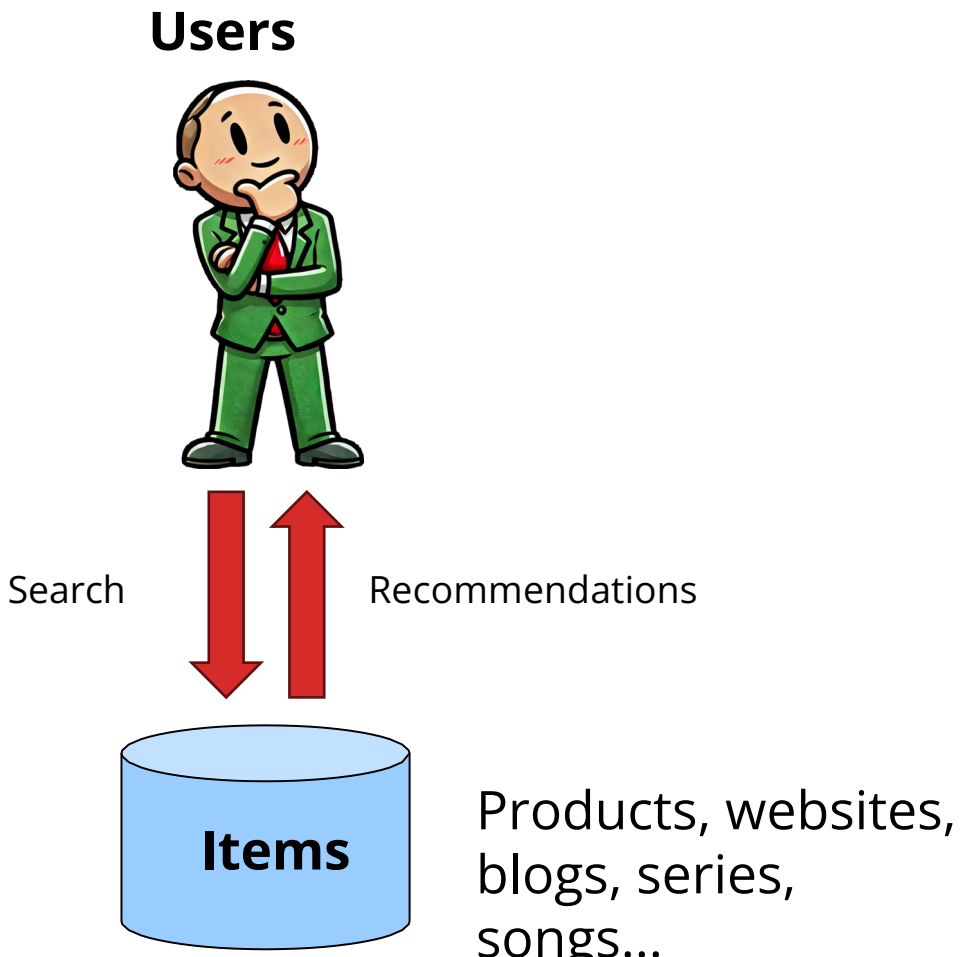
- Buys *hipster* glasses
- Buys beanie hat



Client Y

- Buys *hipster* glasses
- The system recommends beanie hats from the data collected from X

Introduction



Products, websites, blogs, series, songs...

Examples

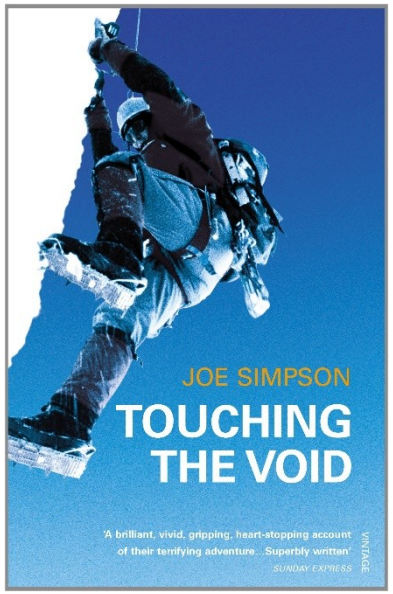
amazon.com · PANDORA · Spotify · NETFLIX · del.icio.us · movielens helping you find the *right* movies · last·fm the social music revolution · Google News · YouTube · XBOX LIVE

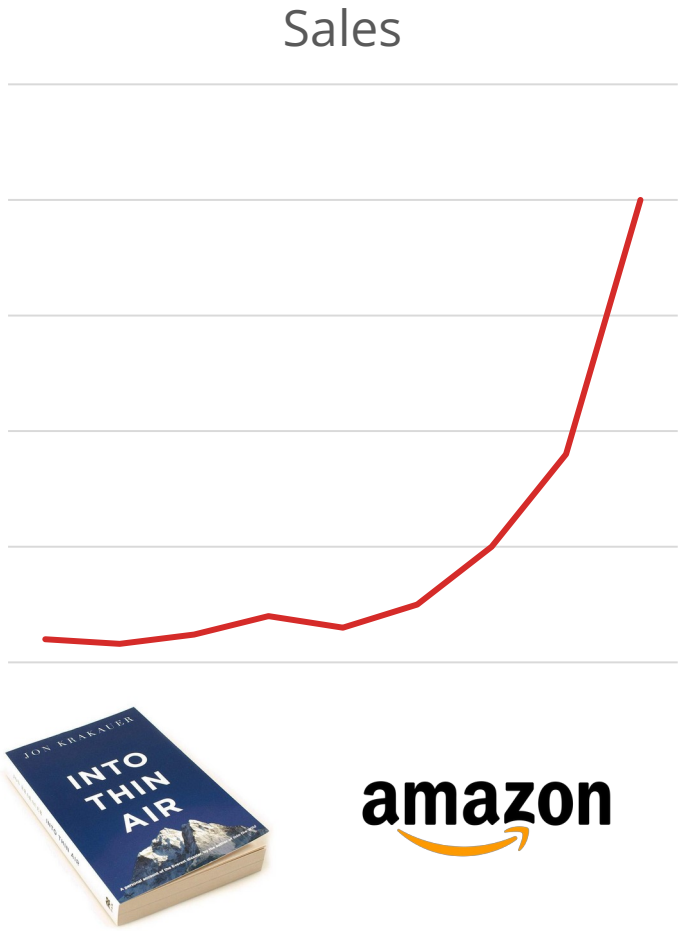
Introduction

- **Types of recommendations**
  - Chosen by groups of experts
    - Lists of “favorites”
    - Lists of "essential" items
  - Simple aggregations
    - Top 10, list of most popular, most recent, etc.
  - Focused on individual users
    - Amazon, Netflix, etc.

Introduction

- **Importance of recommender systems**





Introduction

- **Formal model**
  - U = set of users
  - P = set of items
  - Utility function: $r: U \times P \rightarrow R$
    - R = set of ratings
    - R is totally ordered
      - 0-5 stars
      - A number in [0,1]
      - ...

	Love actually	Bridget Jones	Crazy, stupid, love	X-men	Spider-man
Ane	5	5		1	1
Beñat	5		4	1	1
Clara	1	1		5	4
Daniel	1		1	5	

Introduction

- **Utility matrix**

	Love actually	Bridget Jones	Crazy, stupid, love	X-men	Spider-man
Ane	5	5		1	1
Beñat	5		4	1	1
Clara	1	1		5	4
Daniel	1		1	5	

Introduction

- **Challenges**
  - Extracting known ratings from the matrix
    - Explicitly:
      - Asking users to rate items
      - Crowdsourcing: Paying to tag items
    - Implicitly:
      - Getting ratings from users' actions
        - For example, looking at purchases, interactions…
        - Can work to estimate high ratings, what about low ones?

Introduction

- **Challenges**
  - Extrapolating unknown ratings from known ones
    - We are interested on high ratings
    - Problems:
      - The utility matrix R is sparse
        - Most users rate only a small fraction of the available items
      - Cold-start: New users don't have any ratings
    - **Two approaches**:
      - Content-based recommenders
      - Collaborative filtering
  - Evaluating the performance of the recommender

Outline

- Introduction
- Content-based recommender systems
- Collaborative filtering-based recommender systems
  - User – user
  - Item – item
  - Latent factors
- Evaluation

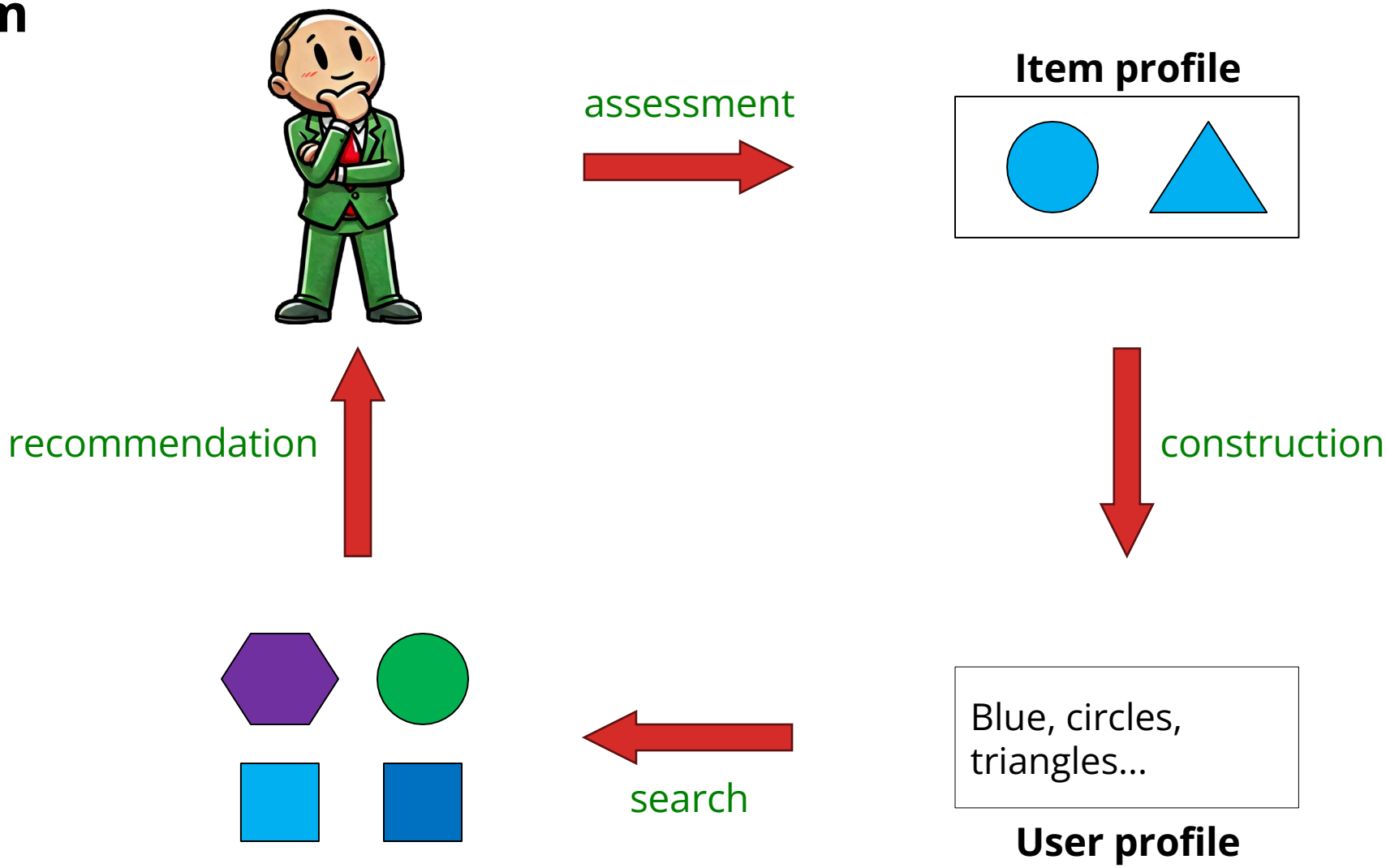
Content-based recommender systems

- **Idea of content-based recommendations**
  - Recommending user ***x*** the items that are similar to the ones that the user has previously rated positively
    - It is based on items' properties
    - Similarity of items = similarity of their characteristics

  - Example
    - Movie recommender system
      - Movies with same cast, same director, genre…
    - Web pages, blogs, news
      - Pages with similar content

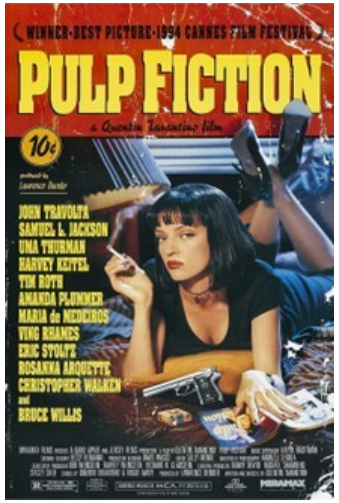
Content-based recommender systems

- **Diagram**



Content-based recommender systems

- **Item profile**
  - We need to create an item profile, for each item (feature representation)
  - Profile = feature vector
    - Movies: author, title, cast, direction, genre…
    - Songs: author, length, style, tempo…
    - Text / images: embeddings (GloVe, word2vec, BERT, CLIP…)



$$\boldsymbol{p} = (p_1, p_2, \ldots, p_d)$$

Content-based recommender systems

- **User profile**
  - We also need to represent users: user profile (feature representation)
  - It is obtained from the item profiles of the items that the user has rated
  - Same number of features as item profiles
  - **Options**:
    - Weighted average of the rated item profiles
    - Weighted average by the deviation of the rated items with respect to the user's mean rating (to avoid users bias in rating scales)
    - …

Content-based recommender systems

- **Predictions: Predicting the rate for an item of a user**
  - Compute the distance / similarity between the item and user profiles
    - Example:
      - For a user, compute the similarity between its profile and the profile of all the movies he hasn't seen (rated)
      - Recommend the one with less distance / more similarity
  - Possible distance / similarity functions:
    - Euclidean distance, Manhattan distance, etc.
    - Cosine similarity:

$$sim_c(\boldsymbol{u}, \boldsymbol{p}) = \frac{\boldsymbol{u} \cdot \boldsymbol{p}}{\left|\left|\boldsymbol{u}\right|\right|_2 \left|\left|\boldsymbol{p}\right|\right|_2}$$

Content-based recommender systems

- **Example** – user profile = weighted avg of ratings for items

	Love actually	Bridget Jones	Crazy, stupid, love	X-men	Spider-man
Ane	5	5		1	1
Beñat	5		4	1	1
Clara	1	1		5	4
Daniel	1		1	5	

Content-based recommender systems

- **Example – weighted avg of items**
  - Suppose an expert committee has assessed each movie based on 3 features:
    - Romance (p_1)
    - Comedy (p_2)
    - Action (p_3)

	Love actually	Bridget Jones	Crazy, stupid, love	X-men	Spider-man
Ane	5	5		1	1
Beñat	5		4	1	1
Clara	1	1		5	4
Daniel	1		1	5	

	Romance (p_1)	Comedy (p_2)	Action (p_3)
Love actually	0.95	0.80	0.01
Bridget Jones	0.90	0.90	0.10
Crazy, stupid, love	0.85	0.99	0.20
X-men	0.10	0.20	0.90
Spider-man	0.10	0.20	0.90

Content-based recommender systems

- **Example – weighted avg of items**
  - First, compute the user profiles
    - Ane
    - Beñat
    - Clara
    - Daniel

	Love actually	Bridget Jones	Crazy, stupid, love	X-men	Spider-man
Ane	5	5		1	1
Beñat	5		4	1	1
Clara	1	1		5	4
Daniel	1		1	5	

	Romance (p_1)	Comedy (p_2)	Action (p_3)
Love actually	0.95	0.80	0.01
Bridget Jones	0.90	0.90	0.10
Crazy, stupid, love	0.85	0.99	0.20
X-men	0.10	0.20	0.90
Spider-man	0.10	0.20	0.90

- **Example – weighted avg of items**
  - First, compute the user profiles
    - Ane

	Love actually	Bridget Jones	Crazy, stupid, love	X-men	Spider-man
Ane	5	5		1	1

	Romance (p_1)	Comedy (p_2)	Action (p_3)
Love actually	0.95	0.80	0.01
Bridget Jones	0.90	0.90	0.10
Crazy, stupid, love	0.85	0.99	0.20
X-men	0.10	0.20	0.90
Spider-man	0.10	0.20	0.90

$$u_A = \frac{1}{5+5+1+1}\left(5p_{LA} + 5p_{BJ} + 1p_{XM} + 1p_{SM}\right)$$

$$u_A = \frac{1}{12}\left(5(0.95, 0.8, 0.01) + 5(0.9, 0.9, 0.1) + (0.1, 0.2, 0.9) + (0.1, 0.2, 0.9)\right) = (0.7875, 0.7417, 0.1958)$$

Content-based recommender systems

- **Example – weighted avg of items**
  - First, compute the user profiles
    - Ane
    - Beñat
    - Clara
    - Daniel

	Love actually	Bridget Jones	Crazy, stupid, love	X-men	Spider-man
Ane	5	5		1	1
Beñat	5		4	1	1
Clara	1	1		5	4
Daniel	1		1	5	

$$u_A = (0.7875, 0.7417, 0.1958)$$

$$u_B = (0.7591, 0.7600, 0.2409)$$

$$u_C = (0.2500, 0.3182, 0.7464)$$

$$u_D = (0.3286, 0.3986, 0.6729)$$

	Romance (p_1)	Comedy (p_2)	Action (p_3)
Love actually	0.95	0.80	0.01
Bridget Jones	0.90	0.90	0.10
Crazy, stupid, love	0.85	0.99	0.20
X-men	0.10	0.20	0.90
Spider-man	0.10	0.20	0.90

Content-based recommender systems

- **Example – weighted avg of items**
  - Then, compute similarities
    - For example, Ane and Daniel

$$sim_c(\boldsymbol{u}, \boldsymbol{p}) = \frac{\boldsymbol{u} \cdot \boldsymbol{p}}{||\boldsymbol{u}||_2 ||\boldsymbol{p}||_2}$$

	Love actually	Bridget Jones	Crazy, stupid, love	X-men	Spider-man
Ane	5	5		1	1
Beñat	5		4	1	1
Clara	1	1		5	4
Daniel	1		1	5	

$$sim_c(Ane, Love\ Actually) = 0.9839$$
$$sim_c(Ane, Bridget\ Jones) = 0.9945$$
$$sim_c(Ane, CrazyStupidLove) = 0.9942$$
$$sim_c(Ane, Xmen) = 0.3956$$
$$sim_c(Ane, Spiderman) = 0.3956$$

$$sim_c(Daniel, Love\ Actually) = 0.6054$$
$$sim_c(Daniel, Bridget\ Jones) = 0.6664$$
$$sim_c(Daniel, Crazy\ Stupid\ Love) = 0.7220$$
$$sim_c(Daniel, Xmen) = 0.9129$$
$$sim_c(Daniel, Spiderman) = 0.9129$$

Content-based recommender systems

- **Pros**:
- No information from other users is needed
- It can make recommendations to users with very particular tastes
- Can recommend new and little-known items
  - It is interpretable

- **Cons**:
- Feature extraction is a complex problem
- Recommendations for new users?
  - Over-specialized: items far from the user profile are not recommended

Outline

- Introduction
- Content-based recommender systems
- Collaborative filtering-based recommender systems
  - User – user
  - Item – item
  - Latent factors
- Evaluation

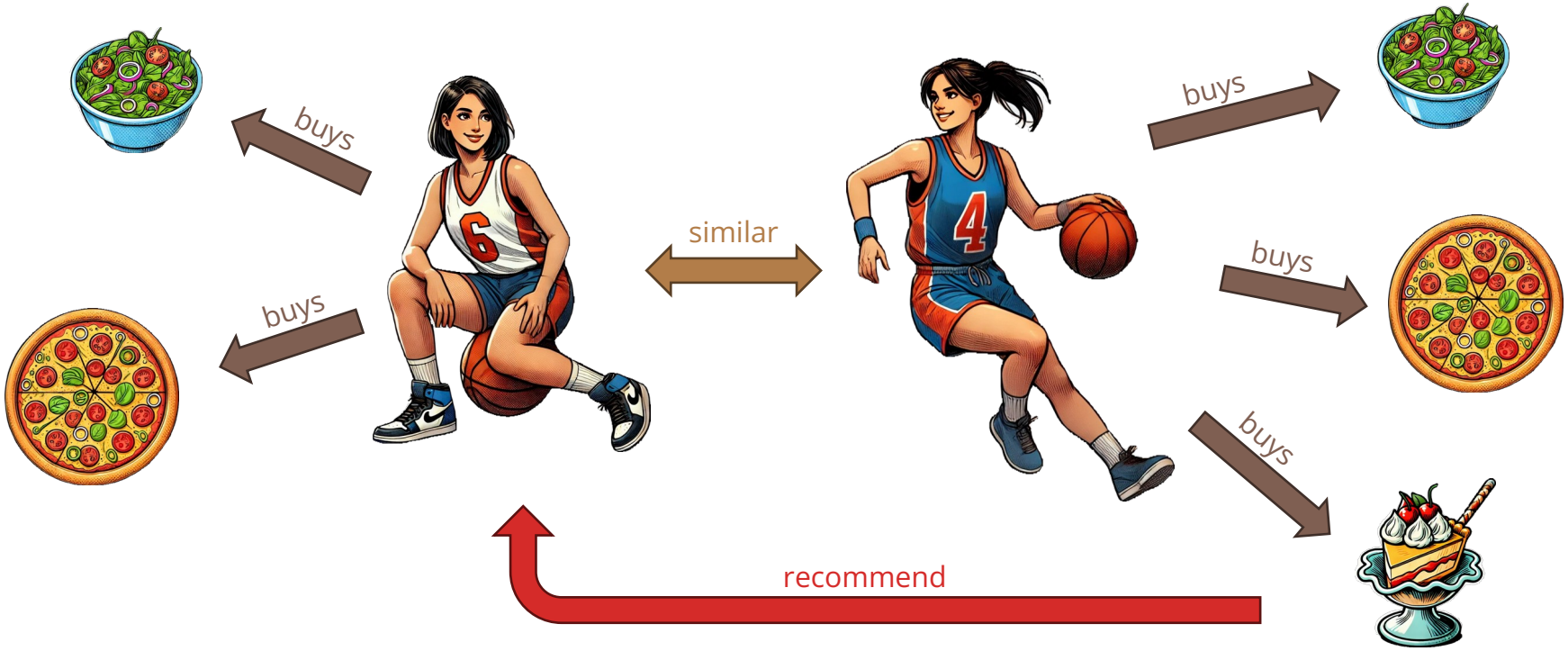
Collaborative filtering

- **Hypothesis**
  - If a user agrees with another user in the opinion of a product, they are more likely to also agree with them, and not with a random user, in the opinion of another product

- **Three types:**
  - User – user
  - Item – item
  - Latent factor models (embeddings)

Collaborative filtering – user – user

- **For a user *u*:**
- Find *k* users with most "similar" ratings to ***u***
  - ***Estimate the ratings of u*** based on the ratings of those *k* users



Collaborative filtering – user – user

- **How to find "similar" users**
  - Set $\boldsymbol{r_u}$ to be the rating vector of user $\boldsymbol{u}$

Ane	5	5		1	1
Beñat	5		4	1	1

    - $r_A = (5,5,0,1,1)$
    - $r_B = (5,0,4,1,1)$

  - Compute cosine similarity
    - Problem: Items without a rating are treated as negative
    - Solution: We subtract from each vector its mean (row-wise)

Note: For data centered on 0, the cosine similarity matches the Pearson correlation coefficient

Collaborative filtering – user – user

- **How to predict ratings**
  - Prediction of the rating of user ***u*** for item ***p***
    - Compute the vector $\boldsymbol{r_u}$
    - Obtain the set *M* of the *k* users most similar to ***u*** who have rated item ***p***
    - We estimate the rating based on the ratings of ***p*** in *M*
  - Alternatives to estimate the ratings
    - Average:

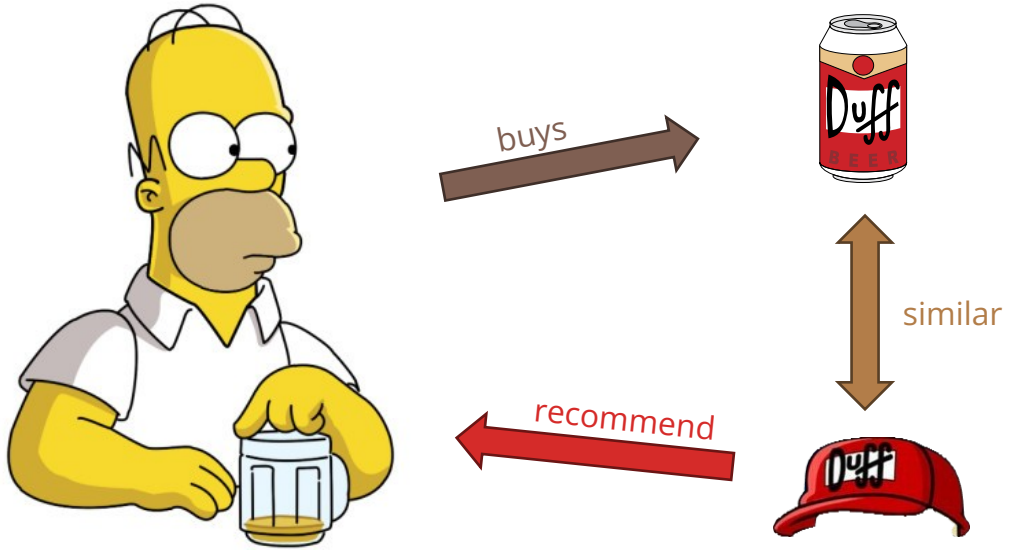
$$\boldsymbol{r_{up}} = \frac{1}{k} \sum_{w \in M} r_{wp}$$

    - Weighted average:

$$\boldsymbol{r_{up}} = \frac{\sum_{w \in M} sim_c(r_u, r_w)\, r_{wp}}{\sum_{w \in M} sim_c(r_u, r_w)}$$

Collaborative filtering – item – item

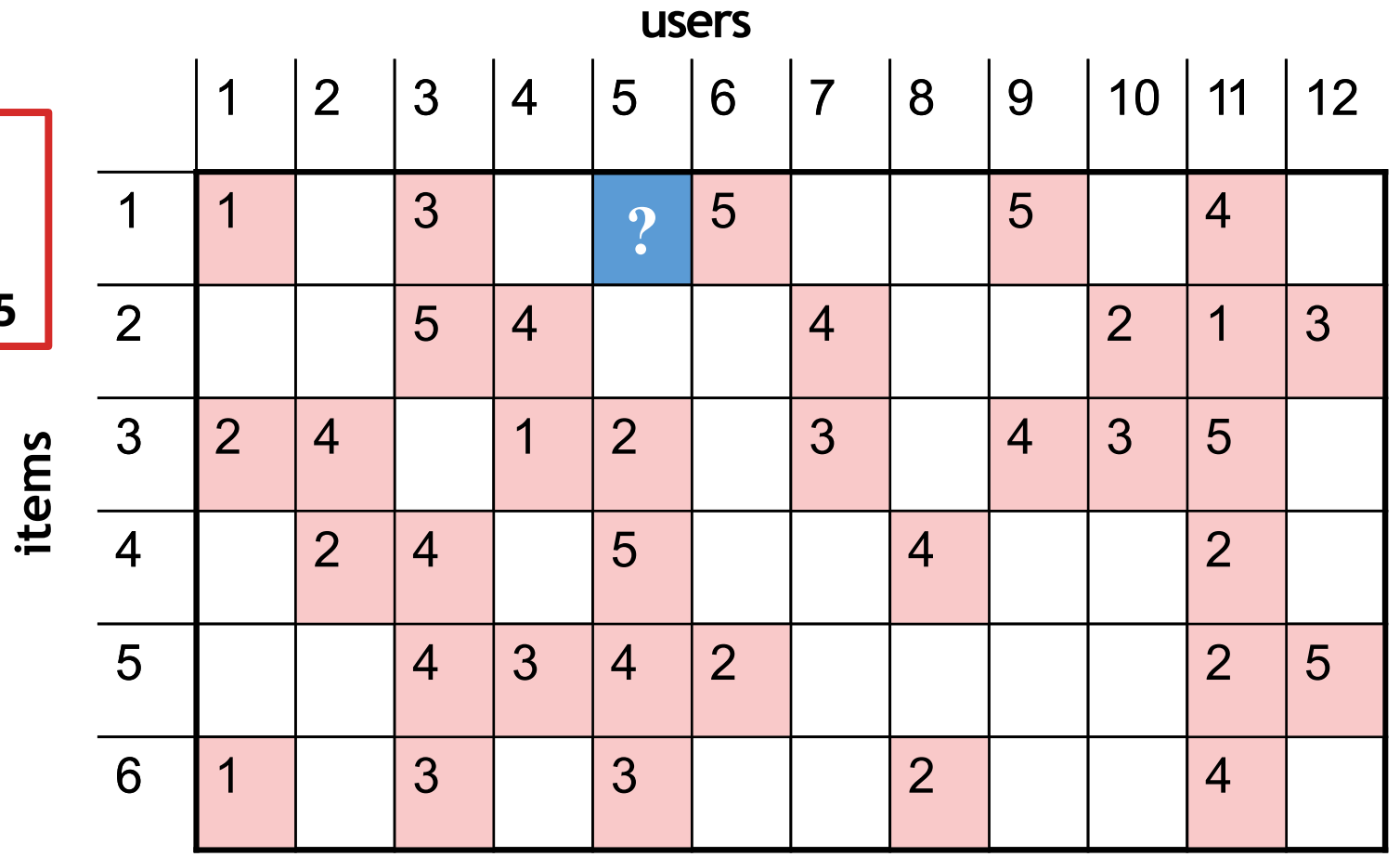
- **For an item *p*:**
- Find the *k* most similar items to ***p***, that have been rated by user ***u***
  - ***Estimate the ratings of p*** based on the ratings of those *k* items



Collaborative filtering – item – item

- **Example – item-item collaborative filtering**

We take the two most similar (k=2) films that **have a rating from user 5**



$sim_c(p_1, p_j)$

1.00

-0.18

0.41

-0.10

-0.31

0.59

Collaborative filtering – item – item

- **Example – item-item collaborative filtering**

users

items	1	2	3	4	5	6	7	8	9	10	11	12	$sim_c(p_1, p_j)$
1	1		3		?	5			5		4		1.00
2			5	4			4			2	1	3	-0.18
3	2	4		1	2		3		4	3	5		0.41
4		2	4		5			4			2		-0.10
5			4	3	4	2					2	5	-0.31
6	1		3		3			2			4		0.59

- Prediction:

$$r_{pu} = \frac{\sum_{q \in Q} sim_c(r_p, r_q)\, r_{qu}}{\sum_{q \in Q} sim_c(r_u, r_q)} = \frac{0.41 \cdot 2 + 0.59 \cdot 3}{0.41 + 0.59} = 2.6$$

Collaborative filtering – item – item

- **Notes**
  - In practice, item-item usually works better than the user-user
    - Normally, the items are "simpler" than the users (multiple likes)
  - No feature extraction required
  - It takes a certain number of users to find neighbors with ratings
  - Items without ratings (new) cannot be recommended
  - Popular items tend to be recommended

- **Collaborative filtering – latent factors**
  - Approach
    - **To learn** from the rating matrix:
      - A feature vector for each user (embedding)
      - A feature vector for each item (embedding)

	Love actually	Bridget Jones	Crazy, stupid, love	X-men	Spider-man
Ane	5	5		1	1
Beñat	5		4	1	1
Clara	1	1		5	4
Daniel	1		1	5	



$$\boldsymbol{u_1} = (u_1^{(1)}, u_2^{(1)}, \dots, u_K^{(1)})$$
$$\boldsymbol{u_2} = (u_1^{(2)}, u_2^{(2)}, \dots, u_K^{(2)})$$
$$\vdots$$
$$\boldsymbol{p_1} = (p_1^{(1)}, p_2^{(1)}, \dots, p_K^{(1)})$$
$$\vdots$$

Collaborative filtering – latent factors

- **Intuition**
  - Suppose we have feature vectors for each movie (as in content-base case)

	Romance (p_1)	Comedy (p_2)	Action (p_3)
Love actually	0.95	0.80	0.01
Bridget Jones	0.90	0.90	0.10
Crazy, stupid, love	0.85	0.99	0.20
X-men	0.10	0.20	0.90
Spider-man	0.10	0.20	0.90

  - We can propose a linear regression model for each user taking into account the movies that each user has rated
    - We will get a feature vector for each user

Collaborative filtering – latent factors

- **Intuition**
  - Suppose we have features p_1, p_2, p_3 as before
  - First, we want to learn user profiles
  - We can propose a linear regression model for each user
    - Ane
    - Beñat
    - Clara
    - Daniel

	Love actually	Bridget Jones	Crazy, stupid, love	X-men	Spider-man
Ane	5	5		1	1
Beñat	5		4	1	1
Clara	1	1		5	4
Daniel	1		1	5	

	Romance (p_1)	Comedy (p_2)	Action (p_3)
Love actually	0.95	0.80	0.01
Bridget Jones	0.90	0.90	0.10
Crazy, stupid, love	0.85	0.99	0.20
X-men	0.10	0.20	0.90
Spider-man	0.10	0.20	0.90

Collaborative filtering – latent factors

- **Intuition**
  - Learning user profiles
    - Ane

	Love actually	Bridget Jones	Crazy, stupid, love	X-men	Spider-man
Ane	5	5		1	1

	Romance (p_1)	Comedy (p_2)	Action (p_3)
Love actually	0.95	0.80	0.01
Bridget Jones	0.90	0.90	0.10
~~Crazy, stupid, love~~	~~0.85~~	~~0.99~~	~~0.20~~
X-men	0.10	0.20	0.90
Spider-man	0.10	0.20	0.90

  - We propose the following linear regression:

$$X = \begin{bmatrix} 0.95 & 0.8 & 0.01 \\ 0.9 & 0.9 & 0.1 \\ 0.1 & 0.2 & 0.9 \\ 0.1 & 0.2 & 0.9 \end{bmatrix} \qquad y = \begin{bmatrix} 5 \\ 5 \\ 1 \\ 1 \end{bmatrix}$$

Collaborative filtering – latent factors

- **Intuition**
  - Learning user profiles
    - Ane

$$u^{(Ane)} = [u_1, u_2, u_3]^T$$

	Romance (p_1)	Comedy (p_2)	Action (p_3)
Love actually	0.95	0.80	0.01
Bridget Jones	0.90	0.90	0.10
~~Crazy, stupid, love~~	~~0.85~~	~~0.99~~	~~0.20~~
X-men	0.10	0.20	0.90
Spider-man	0.10	0.20	0.90

  - We define a linear regression model
  - For a movie $\boldsymbol{p^{(j)}}$, we estimate Ane's rating as follows:

$$h_u(p^{(j)}) = u_0 + (u^{(Ane)})^T p^{(j)}$$

$$X = \begin{bmatrix} 0.95 & 0.8 & 0.01 \\ 0.9 & 0.9 & 0.1 \\ 0.1 & 0.2 & 0.9 \\ 0.1 & 0.2 & 0.9 \end{bmatrix} \quad y = \begin{bmatrix} 5 \\ 5 \\ 1 \\ 1 \end{bmatrix}$$

Note:
- Not every user has rated the same movies
- The training for each user will be done with the information related to the movies they have rated

Collaborative filtering – latent factors

- **Intuition**
  - Learning user profiles
    - After training for Ane, we obtain the following weights:
      - $u_0 = 1.72$
      - $u_1 = 1.66$
      - $u_2 = 2.15$
      - $u_3 = -1.46$

It follows that Ane positively values movies with high values of "romance" and "comedy", but negatively of films with high values of "action"

    - Learning each users' profile:
      - Weights for Ane:
      - Weights for Beñat:
      - Weights for Clara:
      - Weights for Daniel:

1.72	1.66	2.15	-1.46
3.78	2.17	-1.01	-3.11
3.87	-1.45	-1.88	1.28
4.37	-1.66	-2.26	1.38

Collaborative filtering – latent factors

- **Intuition**
  - Thus, the predicted ratings are:

	Love actually	Bridget Jones	Crazy, stupid, love	X-men	Spider-man
Ane	5	5	4,96	1	1
Beñat	5	4,51	4	1	1
Clara	1	1	1,03	5	4
Daniel	1	0,98	1	5	4,99

Collaborative filtering – latent factors

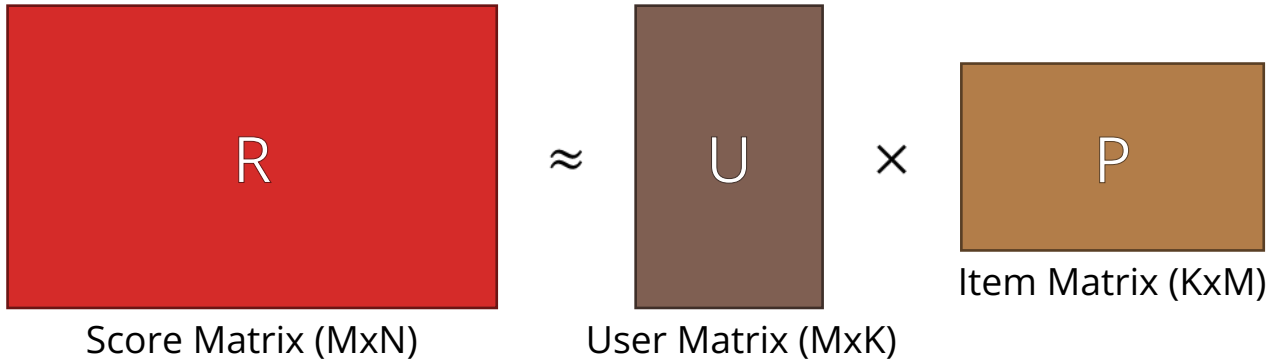
- Actually, in **collaborative filtering with latent factors**
  - Both embedding vectors are learned: user and item vectors
    - We represent each user with a vector of K components
    - We represent each item with a vector of K components
      - These are called latent factors or embeddings

  - As before, the prediction for **user i's rating of the movie j** will be the **scalar product** between the vector representing the user i and the vector representing the film j.

  - This process can be seen as a matrix factorization

Collaborative filtering – latent factors

- **Learning the embeddings**



  - We want to reduce the difference between real ratings and predicted ones

$$\min_{U,P} \|R - U \times P\|_2 + \lambda_1 \|U\|_2 + \lambda_2 \|P\|_2$$

  - How? Optimization algorithm – gradient descent, Adam…

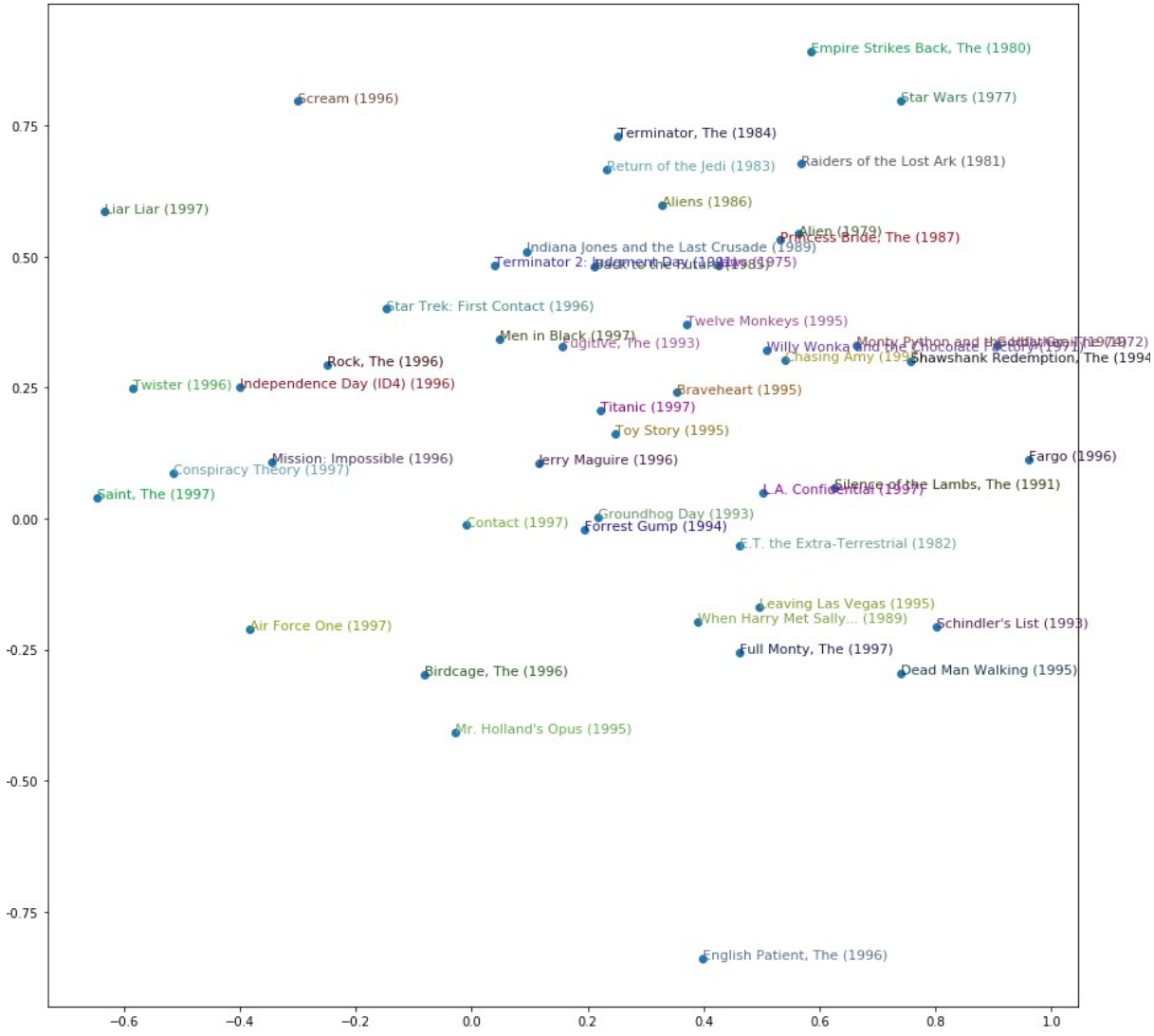
Collaborative filtering – latent factors

- **Problem**
  - We have overlooked one key issue:
    - Not all users rate movies with the same criteria
    - There are more (and less) acclaimed movies in general
  - **Solution**: Including a bias term for each user and for each movie
    - The prediction for **user i's rating of the movie j** will be:

$$\hat{r}^{(i,j)} = u^{(i)T} p^{(j)} + u_0^{(i)} + p_0^{(j)}$$

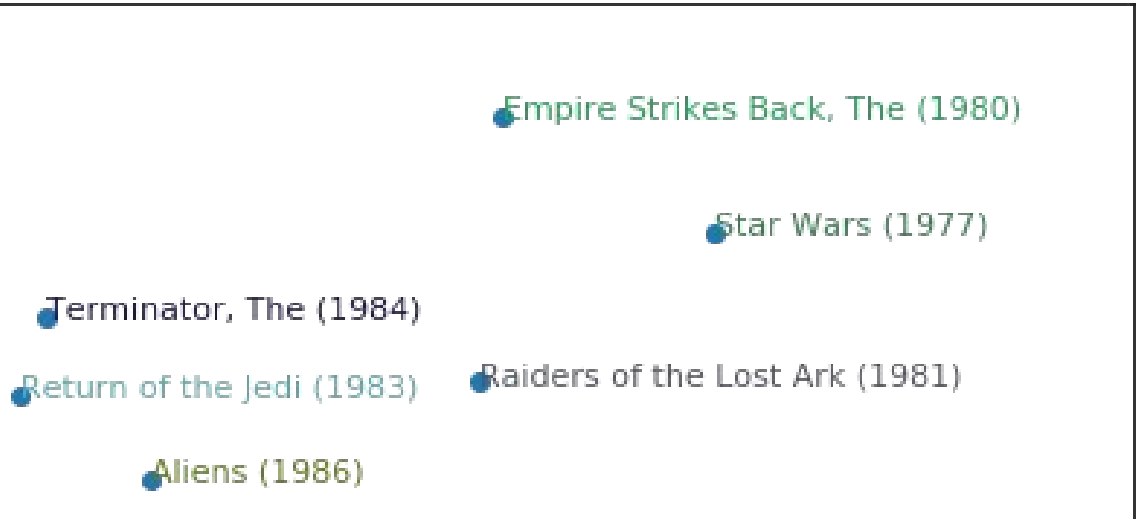
Collaborative filtering – latent factors

- **Embeddings**
  - We can understand embeddings as if they were characteristics of movies (romance, action, etc.)
  - However, they are learned values whose real meaning we do not know
  - K is a hyper-parameter
  - We can calculate similarities between embeddings, perform clusters, etc.



Collaborative filtering – latent factors

- **Embeddings**
  - We can understand embeddings as if they were characteristics of movies (romance, action, etc.)
  - However, they are learned values whose real meaning we do not know
  - K is a hyper-parameter
  - We can calculate similarities between embeddings, perform clusters, etc.



Collaborative filtering – latent factors

- **Cold-start**
  - One of the biggest problems: cold-start
    - Situation where a user or movie has no ratings
  - How do we fix the embeddings?
    - Average embeddings of other users/items
      - Can result in unrealistic embedding
        - For example, a movie with a lot of shots but little action
  - Assign a random embedding
  - Questionnaire to all new users to form their initial embedding

Outline

- Introduction
- Content-based recommender systems
- Collaborative filtering-based recommender systems
  - User – user
  - Item – item
  - Latent factors
- Evaluation

Evalution

- **Evaluation metrics**
  - Like in regression problems
    - MAE, MSE, RMSE, etc.
  - If we can know which of the recommended items are truly accurate, classification metrics can be used
    - Accuracy, precision, recall, fscore, etc.
  - Since we care about the top of the list quality
    - Precision@K: fraction of recommended items in Top-K that are relevant
    - Recall@K: fraction of relevant items we managed to include in Top-K.
  - Ranking metrics
    - NDCG (or NDCG@K), RBP, ERR, etc.

Evalution

- **Partitioning**
  - How do we split data?
    - Train / test
    - Validation sets
    - ...

